

CompNeuroVis Architecture Proposal

Native simulator attachment, plural app modes, and swappable frontends

Purpose. This proposal consolidates the architectural direction discussed for CompNeuroVis: keep native simulator programming as a first-class user experience, while supporting live apps, static/replay visualization, document editors such as a NeuroML editor, notebook frontends, desktop frontends, and future web frontends over a shared scene/update substrate.

Core conclusion. The fundamental architecture is sound if the core remains domain-neutral and the public API is split into host packages and domain integrations. The main correction is that a declarative application DSL should not replace NEURON, NeuroML, or other native authoring models. It should be optional, layered, and generated from lower-level attachment APIs when helpful.

Prepared as an internal architecture proposal for CompNeuroVis.

Contents

1. Executive summary
2. Product goals and non-goals
3. Architectural thesis
4. Proposed layered architecture
5. App modes
6. Package and dependency boundaries
7. Native simulator attachment API
8. Jupyter and standalone frontends
9. NeuroML editor architecture
10. Runtime, commands, updates, and tools
11. Comparative findings from mature systems
12. Implementation roadmap
13. Risks and architectural tests
14. Appendix: source notes

1. Executive summary

CompNeuroVis should be treated as a computational neuroscience application framework, not only as a NEURON viewer and not only as a declarative visualization DSL. Its architectural center should be a stable, domain-neutral substrate that represents data, geometry, views, layout, commands, updates, state, and tools. Multiple authoring modes should sit above that substrate, and multiple frontend/transport packages should sit below it.

The primary architectural correction is this: the public API must not force scientists to stop programming in their native tool. A NEURON user should be able to write ordinary NEURON code, then attach CompNeuroVis traces, controls, morphology views, and tools. A NeuroML user should be able to open and edit a NeuroML document. A replay user should be able to load saved arrays and render a scene. A notebook user should be able to import a Jupyter host package and display the same app object without rewriting the model.

Native simulator code remains the source of truth.
CompNeuroVis attaches, observes, controls, edits, and renders.
Higher-level builders are optional conveniences, not replacements for native authoring models.

This architecture is consistent with mature systems. Matplotlib and ipywidgets layer over ordinary Python objects. Blender add-ons extend Blender data-blocks rather than replacing them. Unity and Unreal preserve native object/component models while adding inspectors, input abstractions, packages, and editor tooling. VTK and ParaView separate data/pipeline state from views and remote execution. OpenMM, Arbor, Brian2, NEST, and LFPy show different ways to separate core execution from higher-level wrappers, monitors, probes, and application layers.

2. Product goals and non-goals

2.1 Goals

- Let users program at the layer they already know, especially NEURON users who want to keep writing NEURON code.
- Support multiple app modes: native simulator attachment, live interactive simulation, static/replay visualization, document editing, hybrid editor-plus-simulation preview, and headless/export workflows.
- Support multiple hosts: standalone desktop, Jupyter notebooks, and eventually browser/web deployment.
- Preserve swappable backends, frontends, and transport layers without requiring every app author to interact with low-level scene/update plumbing.
- Make domain integrations - such as NEURON and NeuroML - siblings that lower into a shared core rather than subclasses of one another.
- Reduce boilerplate in examples without hiding native simulator semantics from expert users.

2.2 Non-goals

- CompNeuroVis should not become a replacement model-description language for NEURON, Jaxley, NeuroML, MOOSE, or OpenMM.
- CompNeuroVis should not make every app pretend to be a live simulator session.
- CompNeuroVis should not expose FieldAppend, FieldReplace, PanelSpec, ViewSpec, and low-level layout mutation as the ordinary authoring surface for scientific users.
- CompNeuroVis should not make the Qt/VisPy desktop frontend the root ontology of the entire framework.
- CompNeuroVis should not require notebook users to install desktop GUI dependencies when they only want notebook rendering.

3. Architectural thesis

The architecture should be multi-entry, single-substrate. Different authoring styles should converge into a common internal representation, but users should approach the framework through APIs that match their task.

```

NEURON code path      \
NeuroML editor path   \
Replay/static viewer path  -> CompNeuroVis core IR -> frontend / transport
Custom Python app path /
Headless export path  /

```

The key distinction is between the internal representation and the public authoring surfaces. Field, Geometry, Scene, ViewSpec, PanelSpec, LayoutSpec, Command, Update, StateStore, and Tool are good internal abstractions. They are not always good public abstractions. A NEURON user wants to say "trace this ref" or "control this mechanism parameter." A NeuroML editor user wants to open, validate, inspect, edit, and export a document. A replay user wants to load data and show it. These all lower to the same substrate, but they should not require the same user code.

4. Proposed layered architecture

Layer	Responsibility	Examples
Core data/view IR	Frontend-neutral description of data, geometry, views, panels, and layout.	Field, Geometry, Scene, ViewSpec, PanelSpec, LayoutSpec
Runtime/event substrate	State, commands, updates, sessions, tool manager, selection, transport, and cadence control.	Command, Update, StateStore, Tool, Selection, Transport
App mode abstractions	Task-shaped wrappers that decide how users author and run apps.	StaticApp, LiveApp, SimulatorApp, DocumentApp, HybridApp
Domain integrations	Adapters for scientific ecosystems and document formats.	NEURON, NeuroML, Jaxley, MOOSE, OpenMM, custom Python
Host/frontends	Concrete rendering and interaction environments.	standalone Qt/VisPy, Jupyter/ipywidgets, future browser frontend
Convenience templates	Prebuilt apps and examples for common workflows.	single_cell_viewer, network_viewer, pharynx_chain_viewer, neuroml_editor

```

compneurovis-core
  Field, Geometry, Scene, ViewSpec, PanelSpec, LayoutSpec
  Command, Update, StateStore, Tool, Selection, Transport interfaces

compneurovis-neuron
  NeuronViewer, NeuronLiveApp, SectionGeometryMapper, record refs, IClamp tools

compneurovis-neuroml
  NeuroMLDocument, NeuroMLEditor, validation, property inspectors, export bridges

compneurovis-standalone
  Qt windowing, VisPy 3D host, pyqtgraph plots, local process/session runner

compneurovis-jupyter
  ipywidgets controls, notebook display adapters, optional VisPy-jupyter_rfb, pythreejs, K3D, plotly

```

5. App modes

CompNeuroVis should explicitly name its major app modes. This prevents NEURON live simulation from becoming the implicit root abstraction for everything.

App mode	Primary user mental model	Typical data flow	Examples
StaticApp	Load data and display it.	Data -> Scene -> Frontend	Saved voltage traces, morphology snapshots, static graph layouts
LiveApp	Run a live mutable app with fields and controls.	Runtime tick -> updates -> frontend	Interactive trace viewer, parameter sweep UI
SimulatorApp	Attach to a native simulator.	Simulator objects -> adapter -> scene/runtime	NEURON, Jaxley, MOOSE, OpenMM
DocumentApp	Edit a persistent document with validation and undo.	Document -> edit commands -> document/scene patches	NeuroML editor, mechanism editor, metadata editor

HybridApp	Edit a document and preview generated simulation behavior.	Document -> generated simulator model -> live preview	NeuroML editor with NEURON preview
HeadlessExport	Produce artifacts without interactive UI.	Scene/model -> PNG/PDF/HTML/JSON/export	Batch figures, shareable scenes, static reports

6. Package and dependency boundaries

The package boundary should match user intent and dependency cost. Notebook users should not need Qt. Desktop users should not need notebook widgets. NEURON integration should not be tangled with the desktop frontend. NeuroML editing should not require NEURON unless simulation preview is enabled.

Distribution / extra	Required dependencies	Should not depend on	Purpose
compneurovis-core	numpy, lightweight schema/data tools	Qt, VisPy, ipywidgets, NEURON	Shared IR, protocol, app/session interfaces
compneurovis-neuron	compneurovis-core, neuron, numpy	Qt, ipywidgets	Native NEURON attachment and simulator-specific tools
compneurovis-neuroml	compneurovis-core, NeuroML-related libraries	Qt, NEURON unless preview enabled	Document model, editor helpers, validation, export bridges
compneurovis-standalone	compneurovis-core, Qt binding, VisPy, pyqtgraph	ipywidgets	Desktop host and local live runner
compneurovis-jupyter	compneurovis-core, ipywidgets	Qt, pyqtgraph	Notebook host and notebook display adapters
Optional rendering extras	vispy[jupyter_rfb], pythreejs, K3D, plotly	Required by core	Notebook 3D and plotting backends

A monorepo with optional extras is probably the best early implementation strategy. Separate distributions can come later if dependency conflicts or install size become painful.

```

pip install compneurovis[standalone]
pip install compneurovis[jupyter]
pip install compneurovis[neuron,jupyter]
pip install compneurovis[neuroml,standalone]

```

7. Native simulator attachment API

The native simulator attachment API is the most important public surface for expert computational neuroscience users. It should let users write normal simulator code and then register visualization, controls, traces, and tools.

```

from neuron import h
from compneurovis_neuron import NeuronViewer
from compneurovis_jupyter import display_app

# Normal NEURON code remains the source of truth.
soma = h.Section(name="soma")
soma.insert("hh")
stim = h.IClamp(soma(0.5))
stim.delay = 5
stim.dur = 1
stim.amp = 0.1

# CompNeuroVis attaches on top.
viewer = NeuronViewer(title="Single cell")
viewer.add_sections([soma])
viewer.trace("soma.v", soma(0.5)._ref_v, unit="mV")
viewer.slider("stim.amp", target=stim, attr="amp", min=0.0, max=1.0)

display_app(viewer)

```

7.1 What users should not have to write

Users should not normally instantiate `Field`, `FieldAppend`, `FieldReplace`, `LinePlotViewSpec`, `PanelSpec`, and layout patches for common workflows. Those should be generated by trace/control/tool registration.

```
# Too low-level for normal app authors:
scene.fields[...] = Field(...)
scene.views[...] = LinePlotViewSpec(...)
self.emit(FieldAppend(...))
self.emit(FieldReplace(...))
scene.layout.replace_panels(...)
```

7.2 Application to the current pharynx example

The current pharynx example is valuable because it exposes what the framework needs to compress. It simultaneously builds a NEURON model, defines recorder refs, creates fields and views, mutates layout, defines controls, applies controls, manages IClamp modes, handles entity clicks, emits reset updates, and prints parameters. This does not imply the core is wrong. It implies the attachment API should own trace/control/tool boilerplate.

8. Jupyter and standalone frontends

The host package should be a thin decision about where and how an app is rendered. A domain app object should not care whether it is displayed in a standalone desktop window or inside a notebook.

```
app = build_some_compneurovis_app(...)

# Desktop host
from compneurovis_standalone import run_app
run_app(app)

# Notebook host
from compneurovis_jupyter import display_app
display_app(app)
```

8.1 Notebook feasibility

Notebook support is feasible in tiers. The easiest path is a notebook display wrapper for scenes, controls, and line plots. A more complete path uses ipywidgets for controls and either VisPy with `jupyter_rfb`, `pythreejs`, `K3D`, or `plotly` for graphics. Full picking and tool parity should be a later milestone.

Capability	Estimated difficulty	Notes
Static scene display	Low	Map <code>Scene/Field/ViewSpec</code> to notebook widgets or plots.
ipywidgets controls	Low	<code>ControlSpec</code> maps naturally to sliders, buttons, dropdowns, and checkboxes.
Live line plots	Moderate	<code>FieldAppend</code> is a good protocol shape; throttle updates for notebook comms.
Same-kernel live NEURON stepping	Moderate	Works if update cadence is decoupled from simulation step cadence.
VisPy inline 3D	Moderate	Use notebook-compatible backends such as <code>jupyter_rfb</code> where practical.
Entity clicking and picking	Moderate to high	Feasible but backend-specific. Not a first milestone.
Full desktop layout parity	High	Notebook layout constraints differ from Qt splitters and dock-like panes.
Full browser-native WebGL frontend	High	Longer-term path for polished web/notebook rendering.

9. NeuroML editor architecture

A NeuroML editor is not just a live simulator with panels. It is a document application. It needs document state, property inspection, schema-aware validation, undo/redo, file import/export, graph/morphology views, and optionally live simulation preview. Therefore it should use DocumentApp or HybridApp, not NeuronSession as the root abstraction.

```
editor = DocumentApp(document=NeuroMLDocument.Load(path))
editor.add(MorphologyEditor())
editor.add(ChannelMechanismInspector())
editor.add(ConnectivityGraphView())
editor.add(ValidationPanel())
editor.add(ExportAction("to_neuron"))
editor.run()
```

9.1 Editor-specific updates

The command/update protocol should be extensible beyond live trace updates. A document editor needs semantic updates such as document patches, validation changes, selection changes, property changes, graph changes, preview rebuilds, and export requests.

```
class Update: ...
class FieldAppend(Update): ...
class FieldReplace(Update): ...
class ScenePatch(Update): ...
class DocumentPatch(Update): ...
class ValidationUpdate(Update): ...
class ToolStateUpdate(Update): ...
```

10. Runtime, commands, updates, and tools

The current typed update strategy is a strength. It should be generalized, not discarded. FieldAppend is good for live traces. FieldReplace is good for replacement data. ScenePatch and layout patches are good for structural UI changes. DocumentPatch and ValidationUpdate should be added for editor applications. The runtime substrate should own cadence, throttling, event routing, state synchronization, and tool modes.

10.1 Tools as transient interaction logic

Tools should be modeled as transient interaction logic over stable scene, state, and document/simulator objects. This is closer to Blender modal operators, VTK interactor styles, and Unreal input contexts than to persistent model data. For example, IClamp placement is not part of the morphology itself. It is a tool that interprets clicks, creates simulator objects, and updates session state.

Tool type	State it may own	Targets it may modify
IClampPlacementTool	mode, amplitude mode, selected targets	NEURON IClamp objects and selection state
TraceSelectionTool	selected entity IDs	StateStore selected_trace_entity_ids
NeuroMLPropertyEditorTool	active selection, edit focus	NeuroML document patch stream
AblationTool	target mode, selected cells/segments	Simulator state or generated perturbation config
CameraTool	camera transform, orbit/pan/zoom state	Frontend view state

11. Comparative findings from mature systems

The following findings from comparable frameworks remain relevant after the architectural correction toward native simulator attachment and multiple app modes.

System	What it does architecturally	Lesson for CompNeuroVis
Matplotlib	Layers plotting over arbitrary user-computed arrays.	Do not own the scientific model. Provide simple attachment and rendering functions.
ipywidgets	Binds Python-side state to frontend controls	Controls should bind to simulator or document state

	through widget models and comms.	without requiring low-level protocol code.
Streamlit / Panel	Wrap ordinary Python code in interactive app surfaces. Panel/Param emphasizes reactive parameters.	Useful inspiration for notebook and app authoring, but live simulation needs finer update control than full-script reruns.
Unity	Users work with GameObjects, Components, Prefabs, ScriptableObjects, packages, and input actions.	Separate runtime objects, reusable authored bundles, shared data, packages, and logical inputs.
Unreal	Actors/Components/UObjects are extended by Blueprints, plugins, modules, and Enhanced Input contexts.	Keep a stable native object model while adding higher-level tooling and modular plugins.
Blender	Persistent data-blocks are extended by add-ons, operators, panels, and modal tools.	Separate persistent data from transient tools. Treat editor applications as document apps.
VTK / ParaView	Separate data objects, pipelines, views, state files, programmable filters, and client-server execution.	Keep Field/Geometry/Scene as frontend-neutral state and use reproducible scene/app artifacts.
OpenMM	Separates System/Force/Integrator/Context from Platforms, plugins, serializers, and application wrappers.	Separate core model, execution context, backend platform, plugins, and application convenience layer.
Arbor	Separates recipe from simulation; provides single_cell_model convenience layer.	Expose both low-level extensibility and higher-level wrappers. Do not make convenience wrappers mandatory.
Brian2	Offers high-level equations and monitors while compiling to runtime or standalone device backends.	Make monitors/traces first-class and let backends choose execution details.
NEST	Uses simulator kernel, devices, recording backends, extension modules, and NEST Server.	Recording and transport should be separable framework services.
LFPy	Wraps NEURON with higher-level cell, synapse, electrode, and network abstractions.	Thin domain wrappers can reduce boilerplate while preserving the underlying simulator.

12. Implementation roadmap

Phase	Milestone	Purpose	Success criterion
1	Extract compneurovis-core boundaries	Keep IR/protocol independent of Qt, VisPy, NEURON, and ipywidgets.	Core imports without GUI or simulator dependencies.
2	Build NeuronViewer attachment API	Preserve native NEURON coding while reducing current boilerplate.	Pharynx example can be rewritten without manual FieldAppend/FieldReplace/layout plumbing.
3	Build SceneBuilder / StaticApp	Support replay/static visualization without live simulator assumptions.	Saved arrays and morphology render without NeuronSession.
4	Build compneurovis-jupyter minimal host	Display scenes, controls, and live traces in notebooks.	One-line host swap from standalone to notebook for a simple app.
5	Generalize tool manager	Move click modes, selection, and perturbation tools out of app classes.	IClamp placement is reusable and not hard-coded in pharynx example.
6	Introduce DocumentApp	Support NeuroML editor and other editable model documents.	Open, inspect, validate, patch, and save a NeuroML document.
7	Add HybridApp preview bridge	Support editor plus generated simulator preview.	NeuroML document can generate a preview simulation with CompNeuroVis traces.
8	Stabilize package extras	Make install paths clean for standalone, notebook, NEURON, and NeuroML users.	pip extras work without unnecessary transitive GUI dependencies.

13. Risks and architectural tests

Risk	Failure mode	Mitigation
NEURON path becomes root ontology	NeuroML editor, replay viewer, and future backends inherit live-simulator assumptions.	Make AppRuntime/InteractiveSession domain-neutral and move NeuronSession into integration package.

Public API too low-level	Examples keep requiring FieldAppend, FieldReplace, PanelSpec, StateBinding, and manual reset logic.	Add attachment APIs and mode-specific builders that generate low-level protocol objects.
Notebook frontend tries to reuse Qt window	Dependency and layout mismatch, brittle event-loop behavior.	Build NotebookFrontend as a separate frontend over shared Scene/Update protocol.
DSL replaces native simulator usage	Expert users feel forced to translate NEURON or NeuroML into CompNeuroVis abstractions.	Make declarative builders optional and keep native attachment as the central expert path.
Transport/cadence overcoupled to frontend	High-frequency live updates choke notebooks or remote frontends.	Centralize throttling and batch FieldAppend updates in runtime/session layer.
Editor semantics squeezed into trace protocol	NeuroML editor becomes awkward or fragile.	Add DocumentPatch, ValidationUpdate, and document-specific command types.
Package dependency sprawl	Simple notebook users install Qt, or desktop users install notebook stacks.	Split host and domain integration packages with optional extras.

13.1 Architecture test questions

- Can a NEURON user write ordinary NEURON code and add CompNeuroVis in fewer than ten lines?
- Can the same app object be displayed with compneurovis-standalone and compneurovis-jupyter?
- Can static data render without constructing a simulator session?
- Can a NeuroML document be opened and edited without importing NEURON?
- Can Field, Geometry, Scene, ViewSpec, and PanelSpec remain useful across NEURON, NeuroML, replay, and notebook apps?
- Can tools such as IClamp placement be reused across examples without copying mode/click/key handling code?
- Can a high-level convenience template be removed without making the lower-level native attachment API unusable?

14. Appendix: source notes

These source notes reflect the relevant findings from the prior comparative research. They are included to anchor the architecture proposal, not as exhaustive documentation of each framework.

Source family	Relevant finding
CompNeuroVis core model docs	Repository docs describe Field, Geometry, Scene, Session, Frontend, view/layout, and typed session updates.
Unity documentation	GameObject/Component, Prefab/Prefab Variant, ScriptableObject, package, serialized object binding, and Input Actions informed the package/tool/input recommendations.
Unreal documentation	Actor/Component/UObject, Blueprint, plugin/module, and Enhanced Input concepts informed the modular host and tool/input recommendations.
Blender documentation	Data-blocks, DNA/RNA, add-ons, operators, and modal operators informed the split between persistent document state and transient tools.
VTK and ParaView documentation	Data/pipeline/view/state and client-server patterns informed the domain-neutral scene/update substrate.
VisPy documentation	Scenegraph, visuals, canvas backends, cameras, and notebook rendering options informed the standalone and Jupyter frontend split.
OpenMM documentation	System/Force/Integrator/Context, Platform, plugins, XML serialization, and application layer informed backend/plugin separation.
Arbor documentation	Recipe/simulation and single_cell_model informed the separation between low-level extensibility and convenience wrappers.
Brian2 documentation	Equation DSL, devices, code generation, and monitors informed first-class trace/monitor concepts.
NEST documentation	Kernel, devices, recording backends, extension modules, and NEST Server informed recording and transport separation.
LFPy documentation and paper	High-level wrappers around NEURON informed the role of thin domain-specific APIs that preserve native simulator semantics.
ipywidgets, Panel, Streamlit, Dash	Notebook controls, reactive parameters, callback graphs, rerun/session-state models

documentation	informed the notebook frontend feasibility analysis.
---------------	--

Representative URLs reviewed during research:

<https://raw.githubusercontent.com/orrenravid1/CompNeuroVis/main/docs/architecture/core-model.md>
<https://raw.githubusercontent.com/orrenravid1/CompNeuroVis/main/docs/architecture/session-protocol.md>
<https://raw.githubusercontent.com/orrenravid1/CompNeuroVis/main/docs/architecture/vispy-frontend.md>
<https://docs.unity3d.com/2020.1/Documentation/Manual/UsingComponents.html>
<https://docs.unity3d.com/6000.4/Documentation/Manual/class-ScriptableObject.html>
<https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/Actions.html>
<https://dev.epicgames.com/documentation/unreal-engine/actors-in-unreal-engine>
<https://dev.epicgames.com/documentation/unreal-engine/blueprints-visual-scripting-in-unreal-engine>
<https://developer.blender.org/docs/features/core/datablocks/>
<https://docs.blender.org/api/current/bpy.types.Operator.html>
<https://book.vtk.org/en/latest/VTKBook/04Chapter4.html>
<https://docs.paraview.org/en/v5.12.0/Tutorials/ClassroomTutorials/pythonAndBatchParaViewAndPython.html>
https://vispy.org/getting_started/index.html
https://docs.openmm.org/7.6.0/developer/02_core_library.html
<https://docs.arbor-sim.org/en/latest/cpp/index.html>
https://brian2.readthedocs.io/en/2.9.0/advanced/how_brian_works.html
<https://nest-simulator.readthedocs.io/>
<https://lfpy.readthedocs.io/en/v2.3.3/classes.html>
<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Low%20Level.html>
<https://panel.holoviz.org/explanation/api/reactive.html>
<https://docs.streamlit.io/develop/concepts/architecture>